Especificação Técnica: Ingestão de Notícias e Eventos Corporativos

Projeto: Assessor de Investimentos do Futuro (Hackathon) Módulo: coleta diária de notícias (InfoMoney) e eventos corporativos (CVM) + resumo gerado por IA Público deste documento: agente de IA que vai codificar a solução. Siga as seções na ordem; os critérios de aceite de cada tarefa definem quando ela está pronta.

1. Contexto e objetivo

A plataforma consolida informações relevantes para um assessor de investimentos (clientes, lembretes, notícias). Este módulo cuida da parte de notícias. Uma vez por dia, ele deve:

1. Coletar as notícias do mercado financeiro brasileiro publicadas nas últimas 24 horas (fonte única: RSS do InfoMoney).
2. Verificar se houve novos eventos corporativos na CVM nas últimas 24 horas (fatos relevantes, comunicados, avisos a debenturistas etc.).
3. Repassar adiante somente o que é novo (notícias e eventos ainda não processados).
4. Acionar uma IA (LLM) para ler esse material e gerar um resumo para o assessor.

Foco: mercado brasileiro, renda variável (ações, FIIs, ETFs) e renda fixa (debêntures, crédito privado, juros, Tesouro).

Restrições: custo zero ou muito baixo, ingestão por feed/arquivo estruturado (sem scraping de páginas HTML de notícias), execução em lote 1x/dia, sem servidor permanente.

2. Escopo

Dentro do escopo:

- Leitura de feeds RSS do InfoMoney.
- Download e leitura do dataset de dados abertos da CVM (documentos periódicos e eventuais, IPE).
- Filtro de janela de 24h, deduplicação e controle de estado (idempotência).
- Extração opcional de texto dos PDFs dos eventos da CVM.

- Chamada ao LLM e geração do resumo (JSON e Markdown).
- CLI para execução manual e agendada, logs e testes.

Fora do escopo (não implementar agora):

- Vínculo entre ativos e carteira dos clientes (outro módulo do time; apenas deixar os campos de entidades citadas prontos para isso).
- Interface web, envio de e-mail ou push.
- Outras fontes de notícia, tempo real, fundos regulados pela CVM (FII, FIDC) além do conjunto de companhias abertas.

3. Fluxo geral

```
[agendador 1x/dia]
   -> python -m app.run_daily
        1. Calcula a janela: fim = agora (America/Sao_Paulo), inicio = fim -
24h
        2. InfoMoney RSS   -> NewsItem[]        (filtra janela, limpa HTML,
deduplica)
        3. CVM IPE (CSV)   -> CorporateEvent[]  (filtra data e categorias)
        4. State store     -> remove itens já processados em execuções
anteriores
        5. Se não sobrou nada: grava payload vazio, loga e encerra SEM chamar
o LLM
        6. (opcional) baixa PDFs dos eventos e extrai texto
        7. Grava data/runs/AAAA-MM-DD/payload.json
        8. Chama o LLM -> Digest (validado por schema)
        9. Grava digest.json e digest.md; só então marca os itens como vistos
no state store
```

Regra crítica: os itens só são marcados como processados depois que o resumo foi gerado com sucesso. Se o LLM falhar, uma nova execução deve reprocessar os mesmos itens.

4. Stack sugerida

- Python 3.11+
- requests (HTTP, com timeout e retry), feedparser (RSS), beautifulsoup4 (limpar HTML)
- pydantic v2 (modelos e validação)
- csv/zipfile da biblioteca padrão (ou pandas, se preferir)

- pypdf (extração de texto de PDF)
- sqlite3 da biblioteca padrão (state store)
- pytest (testes)
- SDK do provedor de LLM escolhido, atrás de uma interface própria (ver tarefa T6)

5. Estrutura do projeto

```
news_module/
  app/
    __init__.py
    config.py          # configurações e variáveis de ambiente
    models.py          # modelos pydantic
    window.py          # cálculo da janela de 24h
    sources/
      infomoney_rss.py # T2
      cvm_ipe.py       # T3
      pdf_text.py      # T4
    state.py           # T5
    pipeline.py        # T7 (orquestra as etapas)
    llm/
      client.py        # interface LLMClient + implementação
      summarizer.py    # prompt, chunking, validação (T6)
    render.py          # Digest -> Markdown
    run_daily.py       # CLI (T7)
  data/                # saídas (runs/AAAA-MM-DD/...) e state/seen.db
  tests/
    fixtures/          # RSS e CSV de exemplo
  requirements.txt
  .env.example
  README.md
```

6. Modelos de dados (app/models.py)

Todos os datetimes devem ser timezone-aware, em America/Sao_Paulo.

```
class NewsItem(BaseModel):
    id: str                # sha256 da URL canônica
    fonte: str             # 'InfoMoney'
    secao: str             # 'mercados' | 'economia' | 'business' | ...
    titulo: str
```

```
    resumo: str             # texto do feed, sem HTML
    url: str
    publicado_em: datetime

class CorporateEvent(BaseModel):
    id: str                 # f'{Protocolo_Entrega}-{Versao}'
    cnpj: str
    empresa: str
    codigo_cvm: str | None
    categoria: str          # ex.: 'Fato Relevante'
    assunto: str | None
    entregue_em: datetime   # se a CVM trouxer só a data, usar 00:00 do dia
    data_referencia: date | None
    link_documento: str
    texto_documento: str | None   # extraído do PDF e truncado
    texto_status: Literal['ok', 'falha', 'nao_tentado']

class IngestionPayload(BaseModel):
    gerado_em: datetime
    janela_inicio: datetime
    janela_fim: datetime
    noticias: list[NewsItem]
    eventos: list[CorporateEvent]

class DigestItem(BaseModel):
    tipo: Literal['noticia', 'evento_corporativo']
    titulo: str
    resumo: str                       # 2 a 3 frases
    relevancia: Literal['alta', 'media', 'baixa']
    classe_ativo: Literal['renda_variavel', 'renda_fixa', 'macro', 'misto']
    entidades_citadas: list[str]      # empresas/ativos citados
EXPLICITAMENTE no texto
    fonte_url: str

class Digest(BaseModel):
    data_referencia: date
    resumo_executivo: str             # até 6 linhas
    itens: list[DigestItem]
    avisos: list[str]                 # ex.: 'texto do PDF indisponível para
X'
```

7. Módulos e tarefas

T1. Configuração e janela de tempo (config.py, window.py)

- Ler configurações de variáveis de ambiente (.env) com valores padrão:
  - TIMEZONE=America/Sao_Paulo, WINDOW_HOURS=24
  - INFOMONEY_FEEDS: lista de URLs (padrão abaixo)
  - CVM_CATEGORIAS: lista (padrão: Fato Relevante, Comunicado ao Mercado, Aviso aos Debenturistas, Aviso aos Acionistas)
  - HTTP_TIMEOUT=30, HTTP_RETRIES=3, USER_AGENT identificável
  - FETCH_PDF_TEXT=true, PDF_MAX_CHARS=6000, PDF_MAX_BYTES=10000000
  - LLM_PROVIDER, LLM_API_KEY, LLM_MODEL, MAX_INPUT_CHARS=60000
  - DATA_DIR=./data
- Feeds padrão do InfoMoney (validar com requisição real na T2):
  - https://www.infomoney.com.br/mercados/feed/
  - https://www.infomoney.com.br/economia/feed/
  - https://www.infomoney.com.br/business/feed/
- window.py: função get_window(now: datetime | None, hours: int) -> (inicio, fim). Aceitar now injetável para testes.

Aceite: testes unitários cobrindo virada de dia, de mês e de ano, e o parâmetro now.

T2. Fonte de notícias: RSS do InfoMoney (sources/infomoney_rss.py)

- Para cada feed: requests.get com User-Agent, timeout e retry com backoff (1s, 2s, 4s); depois feedparser.parse(response.content). Falha em um feed não derruba os demais: logar e seguir.
- Mapear cada entrada para NewsItem:
  - titulo: entry.title; resumo: entry.summary sem HTML (BeautifulSoup get_text, normalizar espaços); url: entry.link.
  - publicado_em: entry.published_parsed (está em UTC) convertido para America/Sao_Paulo. Entrada sem data: descartar e logar.
  - secao: derivada da URL do feed.
  - id: sha256 da URL canônica (remover parâmetros utm_*, fragmento e barra final).
- Manter apenas itens com inicio <= publicado_em <= fim.

- Deduplicar entre feeds pelo id (a mesma matéria pode aparecer em mais de uma seção); ordenar por publicado_em decrescente.
- Função pública: fetch_news(janela_inicio, janela_fim) -> list[NewsItem].

Aceite: teste com fixture de RSS verifica filtro de janela, remoção de HTML, deduplicação e conversão de fuso. Um feed fora do ar não gera exceção.

T3. Fonte de eventos corporativos: CVM IPE (sources/cvm_ipe.py)

Dataset: Cias Abertas: Documentos: Periódicos e Eventuais (IPE), em dados.cvm.gov.br.

- URL (validar no portal antes de fixar): https://dados.cvm.gov.br/dados/CIA_ABERTA/DOC/IPE/DADOS/ipe_cia_aberta_{ANO}.zip
- Baixar o ZIP em memória (requests com timeout/retry), abrir com zipfile e ler o CSV com separador ';' e encoding latin-1 (ISO-8859-1). Não assumir que existe um arquivo por empresa: é um arquivo por ano com todas as companhias.
- Baixar o arquivo do ano de janela_inicio e, se for diferente, também o do ano de janela_fim (virada de ano).
- Colunas esperadas (validar no CSV real e falhar com mensagem clara se faltar alguma): CNPJ_Companhia, Nome_Companhia, Codigo_CVM, Data_Referencia, Categoria, Tipo, Especie, Assunto, Data_Entrega, Tipo_Apresentacao, Protocolo_Entrega, Versao, Link_Download.
- Filtros:
  - Categoria dentro de CVM_CATEGORIAS (comparação sem diferenciar maiúsculas e acentos).
  - Data_Entrega: se vier só a data, aceitar registros com data >= data de janela_inicio (a deduplicação pelo state store evita repetidos); se vier com hora, aplicar a janela exata.
- id = f'{Protocolo_Entrega}-{Versao}'. Uma reapresentação (versão nova) é tratada como evento novo.
- Função pública: fetch_events(janela_inicio, janela_fim) -> list[CorporateEvent] (ainda sem texto do PDF).
- Observação: o CSV é apenas metadado (empresa, categoria, assunto, link). Não contém ticker. Não tentar resolver ticker neste módulo.

Aceite: teste com CSV de fixture (latin-1, separador ';') cobrindo filtro por categoria, filtro por data, virada de ano e reapresentação.

T4. Texto dos PDFs da CVM (sources/pdf_text.py)

- Para cada CorporateEvent, se FETCH_PDF_TEXT=true: baixar Link_Download com timeout, limite de PDF_MAX_BYTES e verificação de Content-Type.
- Extrair texto com pypdf, normalizar espaços e truncar em PDF_MAX_CHARS.
- Se o link não retornar um PDF (por exemplo, página de visualização), se o PDF for escaneado (texto vazio) ou se ocorrer erro: texto_status='falha', texto_documento=None, e seguir só com empresa, categoria e assunto. Nunca interromper o pipeline por causa de um PDF.
- Limitar a concorrência (no máximo 4 downloads simultâneos) e respeitar o timeout.
- Função pública: enrich_with_pdf_text(eventos) -> list[CorporateEvent].

Aceite: testes com PDF de exemplo e com respostas de erro simuladas (HTTP 404, PDF vazio, arquivo grande demais).

T5. Controle de estado e deduplicação (state.py)

- SQLite em DATA_DIR/state/seen.db, tabela seen(id TEXT PRIMARY KEY, tipo TEXT, visto_em TEXT).
- Funções: filter_new(items) -> itens cujo id ainda não existe; mark_seen(items); prune(dias=30).
- mark_seen só é chamado pelo pipeline após o resumo gerado com sucesso (ver seção 3).
- Rodar duas vezes seguidas sem novidades não deve gerar novo resumo nem chamar o LLM.

Aceite: testes de idempotência (segunda execução devolve lista vazia) e de reprocessamento após falha simulada do LLM.

T6. Resumo com IA (llm/client.py, llm/summarizer.py, render.py)

- Interface própria: LLMClient.generate(system: str, user: str) -> str. Uma implementação concreta conforme LLM_PROVIDER/LLM_MODEL/LLM_API_KEY, para trocar de provedor sem mexer na lógica. Incluir um FakeLLMClient para testes.
- summarizer.summarize(payload) -> Digest:
  1. Montar o texto de entrada: cada item numerado, com tipo, título, empresa (se houver), data, URL e texto (resumo da notícia ou trecho do PDF).

2. Se o total ultrapassar MAX_INPUT_CHARS: dividir em blocos, resumir cada bloco em DigestItem[] e fazer uma chamada final para gerar resumo_executivo a partir dos itens.
3. Pedir saída exclusivamente em JSON no formato do modelo Digest. Remover cercas de código antes do parse e validar com pydantic. Em caso de JSON inválido, repetir até 2 vezes incluindo o erro de validação no prompt; se continuar falhando, levantar exceção (o pipeline não marca nada como visto).
4. Preencher avisos quando houver evento com texto_status='falha' ou quando a entrada for truncada.

- Prompt de sistema (usar como base, em português):
  - Você é um assistente de pesquisa de um assessor de investimentos brasileiro. Resuma notícias e eventos corporativos das últimas 24 horas.
  - Use apenas o conteúdo fornecido. Não invente fatos, números, tickers ou empresas. Cite em entidades_citadas somente nomes que aparecem explicitamente no texto.
  - Não faça recomendação de compra ou venda e não projete preços.
  - Para cada item: resumo de 2 a 3 frases, relevância (alta, media ou baixa) para quem aconselha investidores, e classe_ativo (renda_variavel, renda_fixa, macro ou misto). Eventos que afetam crédito de emissores (debêntures, rebaixamento, vencimento antecipado, covenants) devem ser marcados como renda_fixa.
  - Mesclar notícias que tratam do mesmo assunto em um único item, mantendo a URL mais relevante.
  - resumo_executivo: até 6 linhas com o que o assessor precisa saber primeiro.
  - Responda somente com JSON válido, sem texto antes ou depois.
- render.py: gerar digest.md com título e data, resumo executivo, seção Eventos corporativos, seção Notícias (itens ordenados por relevância, com link da fonte) e seção Avisos.

Aceite: testes com FakeLLMClient cobrindo JSON válido, JSON inválido com retry, chunking (entrada grande) e renderização do Markdown.

T7. Orquestração e CLI (pipeline.py, run_daily.py)

- pipeline.run(now=None, window_hours=24, dry_run=False, skip_llm=False) implementa o fluxo da seção 3.
- CLI: python -m app.run_daily [--now ISO8601] [--window-hours N] [--dry-run] [--skip-llm]

    - --dry-run: coleta e filtra, grava payload.json, não chama o LLM e não altera o state store.
    - --skip-llm: igual, mas permite testar a coleta sem custo.
- Saídas em DATA_DIR/runs/AAAA-MM-DD/: payload.json, digest.json, digest.md e run.log.
- Quando não há itens novos: gravar payload vazio, logar 'sem novidades' e sair com código 0, sem chamar o LLM.
- Códigos de saída: 0 sucesso (inclusive sem novidades), 1 falha total (nenhuma fonte respondeu ou erro do LLM), 2 sucesso parcial (alguma fonte falhou, mas o resumo foi gerado).

Aceite: teste de integração com fontes e LLM simulados cobrindo: execução normal, sem novidades, falha de uma fonte, falha do LLM e segunda execução no mesmo dia.

T8. Agendamento e documentação

- Documentar no README duas opções de execução diária: cron (ex.: 0 6 * * * com TZ=America/Sao_Paulo) ou GitHub Actions (atenção: o horário do schedule é em UTC; 06:00 de Brasília = 09:00 UTC; persistir data/state entre execuções via cache ou artefato).
- README com instalação, variáveis de ambiente (.env.example), como rodar e como interpretar as saídas.

8. Requisitos não funcionais

- Idempotência: reexecutar não duplica itens nem resumos.
- Resiliência: falha de uma fonte ou de um PDF não derruba o pipeline; toda requisição HTTP com timeout e retry.
- Observabilidade: logging estruturado (nível INFO por etapa, com contagens: itens lidos, na janela, novos, descartados).
- Segurança: chaves apenas por variável de ambiente; nunca no código nem nos logs.
- Cortesia com as fontes: no máximo uma requisição por feed por execução, User-Agent identificável, respeitar os termos de uso. Armazenar e enviar ao LLM apenas título, resumo do feed e link das notícias (sem copiar o corpo das matérias).
- Custo: nenhuma chamada ao LLM quando não há itens novos; truncar textos para controlar tokens.

9. Premissas a validar durante a implementação

O agente deve confirmar estes pontos com requisições reais e, se algo divergir, ajustar o código e registrar no README:

1. As URLs dos feeds do InfoMoney respondem e trazem data de publicação em cada entrada.
2. A URL do ZIP do IPE da CVM e o nome real do arquivo CSV dentro dele.
3. Nomes exatos das colunas, encoding e separador do CSV da CVM.
4. Formato de Data_Entrega (só data ou data com hora) e frequência real de atualização do arquivo do ano corrente.
5. Se Link_Download aponta direto para um PDF ou para uma página de visualização.

10. Ordem de implementação recomendada

1. T1 (config e janela) e models.py
2. T5 (state store)
3. T2 (InfoMoney) e T3 (CVM) com fixtures de teste
4. T7 com --dry-run e --skip-llm (coleta ponta a ponta sem custo)
5. T4 (PDFs)
6. T6 (LLM, com FakeLLMClient primeiro)
7. T7 completo e T8

11. Definição de pronto

- python -m app.run_daily --dry-run gera payload.json com notícias e eventos reais das últimas 24h.
- python -m app.run_daily gera digest.json (válido pelo schema Digest) e digest.md.
- Segunda execução imediata não chama o LLM e não duplica itens.
- Suite de testes (pytest) passando sem acesso à internet e sem chave de LLM.
- README e .env.example completos.

12. Extensões futuras (não implementar agora)

- Resolver CNPJ para ticker e cruzar entidades_citadas com a carteira dos clientes do assessor.

- Incluir o dataset de documentos de fundos (FII, FIDC) da CVM.
- Adicionar novas fontes (B3, Banco Central, outros veículos) como novos módulos em sources/ com a mesma assinatura fetch_*(inicio, fim).